

FRANCO PAY

Mini Financial Platform

Full-Stack Development • House of Practice, Round One

Track	02 - Full-Stack Development
Challenge	Mini Financial Platform
Project Type	Financial Transaction Prototype
Developer	Francis Quayson
Version	1.0
Development Period	72 Hours

Project Documentation

Table of Contents

1. Project Overview
2. Problem Addressed
3. Project Objectives
4. Project Scope
5. Technologies and Tools
6. System Architecture
7. Core Functionality
8. Security and Reliability
9. Database Structure
10. API Documentation
11. Transaction Integrity
12. Testing and Validation
13. Technical Decisions
14. Challenges Encountered
15. Known Limitations
16. Answer to the Challenge Question
17. Final Reflection
18. Conclusion
19. Project Declaration

1. Project Overview

FRANCO PAY is a mini financial platform prototype developed for House of Practice, Round One, Track 02 Full-Stack Development. The system demonstrates how a digital financial application can handle user accounts, wallets and money transfers with a stronger focus on security and transaction integrity than a normal CRUD application.

Users can create accounts, log in, view their wallet balance, initiate transfers, confirm recipients and review transaction activity. The project is a prototype and is not intended to operate as a production banking or payment service.

2. Problem Addressed

Beginner financial applications often treat money movement as a simple database update. That approach can create serious problems when requests are repeated, balances are insufficient, users are unauthorized, or a transaction fails during processing.

- Unauthorized access to financial information or actions.
- Transfers being submitted more than once.
- Transfers being attempted with invalid amounts.
- Transfers to invalid or nonexistent recipients.
- Self-transfers that should not be allowed.
- Insufficient funds.
- Inconsistent sender and recipient balances.
- Limited traceability of financial activities.

FRANCO PAY addresses these concerns through authentication, authorization, validation, duplicate-request protection, atomic transaction processing, transaction status handling and audit logging.

3. Project Objectives

- Provide secure user registration and login.
- Create a wallet for each registered user.
- Generate and display a wallet number.
- Allow users to view wallet details and balance.
- Allow users to initiate transfers to valid recipients.
- Confirm recipient details before completing a transfer.
- Reject invalid, unsafe or unauthorized transactions.
- Prevent duplicate transaction requests.
- Maintain transaction history and details.
- Provide a usable dashboard for financial activity.

4. Project Scope

4.1 Included

- User registration.
- Login and logout.
- Wallet creation and wallet number.
- Wallet balance display.
- Money transfer workflow.
- Recipient validation and confirmation.
- Transaction history and status.
- Input validation and authorization.
- Duplicate-request protection.
- Dashboard and recent activity.
- Simple audit logging.

4.2 Excluded

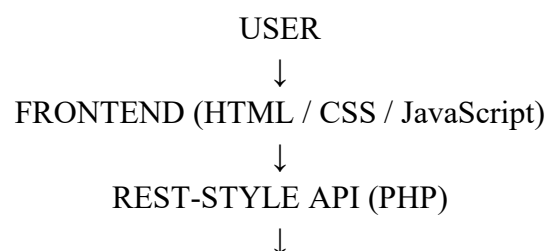
- Real banking operations.
- Real Mobile Money or bank integration.
- Live payment gateway processing.
- Full KYC/AML compliance.
- Biometric authentication.
- Production-grade distributed infrastructure.
- Large-scale financial settlement.

5. Technologies and Tools

Area	Technology / Tool
Frontend	HTML5, CSS3, JavaScript
Backend	PHP 8+ with REST-style API design
Database	SQLite for the local prototype; relational model designed to be MySQL-ready
Development	Visual Studio Code
Runtime	Local PHP server
Version Control	Git / GitHub

6. System Architecture

The system follows a layered architecture:



AUTH • WALLET • TRANSACTION SERVICES



DATABASE (SQLite)

This separation keeps the presentation layer, application logic and persistence layer distinct. It also makes the system easier to test, maintain and extend.

7. Core Functionality

7.1 Authentication

- User registration.
- Password hashing.
- Login and logout.
- Session-based access control.
- Protected API access for authenticated operations.

7.2 Wallet

- Wallet creation for users.
- Wallet number generation.
- Balance retrieval.
- Wallet information displayed on the dashboard.

7.3 Transactions

- Transfer to a valid recipient.
- Recipient details shown for confirmation.
- Validation of transfer amount.
- Self-transfer rejection.
- Insufficient-balance rejection.
- Duplicate-request protection using idempotency.
- Transaction status and history.

7.4 Dashboard

- Account name and greeting.
- Wallet balance.
- Recent activity.
- Quick actions.
- Transaction overview.

8. Security and Reliability

- Passwords are protected using secure password hashing.
- Input is validated before financial operations are processed.
- Authenticated endpoints are protected from unauthorized access.

- Authorization checks ensure users can only perform permitted actions.
- Idempotency keys help prevent repeated requests from creating duplicate transfers.
- Atomic transaction processing is used so related balance changes are handled together.
- Transaction status provides visibility into the outcome of financial operations.
- Audit logging provides traceability for relevant system activities.
- Error handling avoids exposing unnecessary internal system information.

9. Database Structure

The prototype uses a relational data model with the following main entities:

Table / Entity	Purpose	Key Relationship
users	Stores registered user accounts and authentication information.	One user can have a wallet and transactions.
wallets	Stores wallet identity and balance information.	Linked to a user.
transactions	Stores transfer requests, references, amounts and statuses.	Links sender and recipient wallets/users.
ledger_entries	Provides a record of financial entries associated with transactions.	Linked to transactions and wallets.
audit_logs	Records important system and financial activities for traceability.	Linked to relevant users/actions.

10. API Documentation

The application exposes REST-style endpoints for the main operations:

Method	Endpoint	Purpose
POST	/api/auth/register	Create a new user account.
POST	/api/auth/login	Authenticate a user.
POST	/api/auth/logout	End the authenticated session.
GET	/api/wallet	Retrieve the authenticated user's wallet.
POST	/api/transactions	Create/process a transfer request.
GET	/api/transactions	Retrieve transaction history.
GET	/api/transactions/{reference}	Retrieve transaction details by reference.

11. Transaction Integrity

The most important difference between FRANCO PAY and a basic CRUD application is the way financial operations are treated as controlled transactions rather than independent record updates.

20.The sender is authenticated and authorized.

- 21.The recipient is checked for validity.
- 22.The transfer amount is validated.
- 23.Self-transfer is rejected where applicable.
- 24.The sender's available balance is checked.
- 25.Duplicate requests are detected using idempotency protection.
- 26.Related balance changes are processed atomically.
- 27.A failed operation should not leave the system with a partially completed transfer.
- 28.Transaction activity is recorded for traceability.
- 29.The resulting transaction status can be reviewed by the user.

12. Testing and Validation

The system was tested against functional, validation, security and transaction-related scenarios during development.

Test Case	Expected Result	Result
User registration	Valid account is created.	Passed
User login	Valid credentials authenticate the user.	Passed
Wallet retrieval	Authenticated wallet details are returned.	Passed
Valid transfer	Transfer is processed correctly.	Passed
Insufficient funds	Transfer is rejected.	Passed
Self-transfer	Transfer is rejected.	Passed
Duplicate request	Repeated request does not create an unintended duplicate.	Passed
Transaction details	Transaction can be retrieved by reference.	Passed
Authorization	Unauthorized operations are rejected.	Passed
Audit activity	Relevant activity is traceable through audit records.	Passed

For the final submission, screenshots or exported test evidence should be attached to demonstrate the actual application results for the key scenarios above.

13. Technical Decisions

- PHP was selected for the backend because it supports rapid full-stack web development and matches the developer's existing experience.
- SQLite was used for the local prototype to keep the development environment lightweight while maintaining a relational data model.
- A REST-style API separates frontend presentation from backend processing.
- Password hashing was selected instead of storing plaintext passwords.
- Idempotency was included because financial requests must be safe against repeated submissions.
- Atomic transaction processing was prioritized over simple balance updates.
- Transaction references and statuses improve traceability and user visibility.

- The design remains MySQL-ready for future deployment to a larger relational database environment.

14. Challenges Encountered

- Making the dashboard responsive and user-friendly.
- Implementing secure transaction validation.
- Protecting authenticated and authorized operations.
- Handling duplicate financial requests.
- Designing a secure and understandable transfer-confirmation flow.
- Fixing scripts and integrating frontend, backend and database components.
- Testing edge cases that do not normally appear in simple CRUD systems.

15. Known Limitations

- The system is a prototype and is not a production banking platform.
- It does not connect to real banks, Mobile Money providers or payment gateways.
- Full KYC/AML and regulatory controls are outside the project scope.
- The prototype uses SQLite rather than a production database deployment.
- High-availability, distributed processing and production monitoring are not implemented.
- Advanced fraud detection and real-time notifications are outside the current version.

16. Answer to the Challenge Question

What did you do to make your system behave like a financial system rather than a normal CRUD application?

I treated money movement as a controlled financial transaction instead of a simple database update. The system authenticates and authorizes users, validates transfer requests, checks balances, validates recipients, rejects self-transfers, protects against duplicate requests through idempotency, processes related balance changes atomically, records transaction status and maintains audit information. These controls are intended to prevent the system from creating inconsistent balances or unintended duplicate transfers when users or the network behave unexpectedly.

17. Final Reflection

This project changed the way I think about financial software. A financial platform cannot simply focus on whether a feature works under normal conditions. It must also protect identity, validate every request, preserve transaction integrity, prevent duplicate operations and provide traceability when something goes wrong.

Building FRANCO PAY gave me practical experience combining frontend development, backend APIs, database design, authentication, authorization and transaction safety in one system.

If I had another 72 hours, I would...

...add stronger automated testing, improve concurrency handling, introduce more detailed audit and ledger controls, add role-based administration, improve monitoring and prepare the system for secure deployment with a production-grade database and payment integrations.

18. Conclusion

FRANCO PAY is a functional financial transaction prototype that combines frontend development, backend API design, relational data management, authentication, authorization and transaction safety. The project demonstrates that financial applications require stronger controls than ordinary CRUD systems because incorrect or duplicated operations can directly affect balances.

The prototype provides a foundation that can be extended into a more complete fintech platform with production infrastructure, external payment integrations, stronger compliance controls, automated testing and advanced security features.

19. Project Declaration

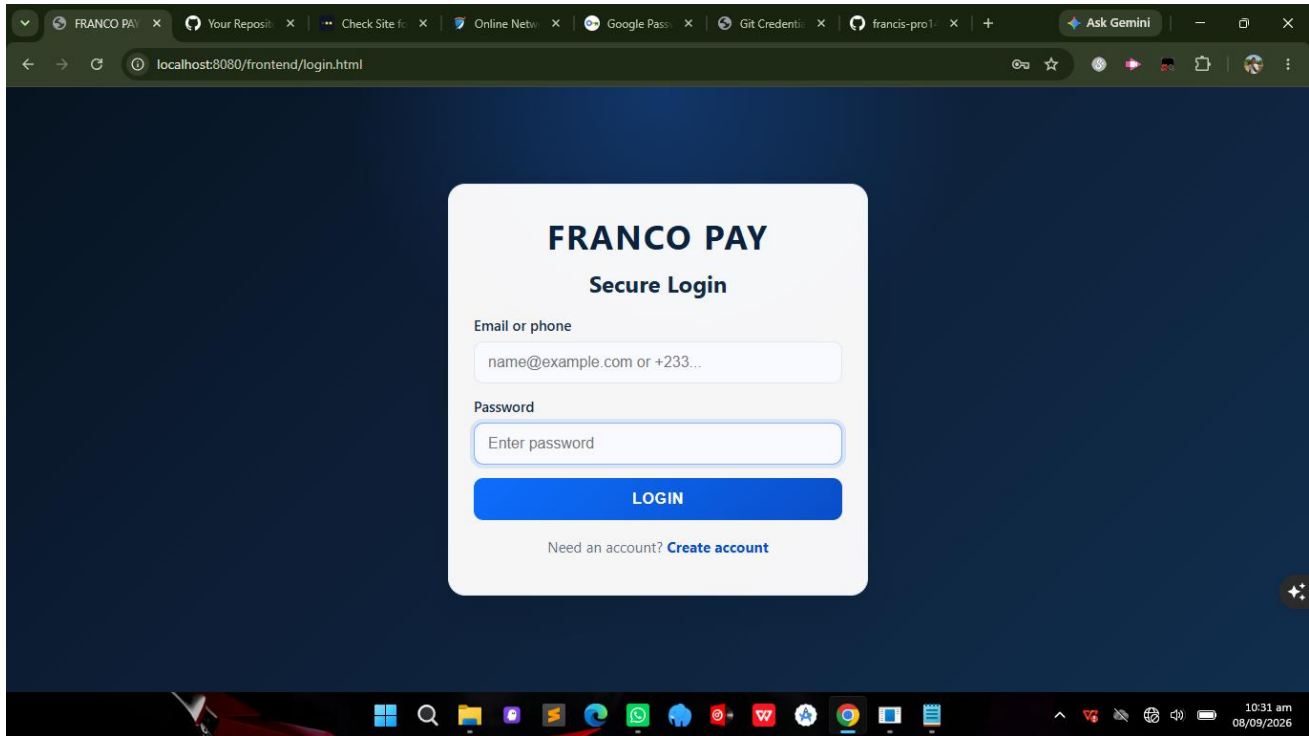
I declare that FRANCO PAY was developed as an independent prototype for the House of Practice, Round One challenge. The implementation, technical decisions and limitations presented in this document are intended to accurately describe the project and its development objectives.

Developer	Francis Quayson
Project	FRANCO PAY — Mini Financial Platform
Version	1.0
Code repository	https://github.com/francis-pro142/franco-pay

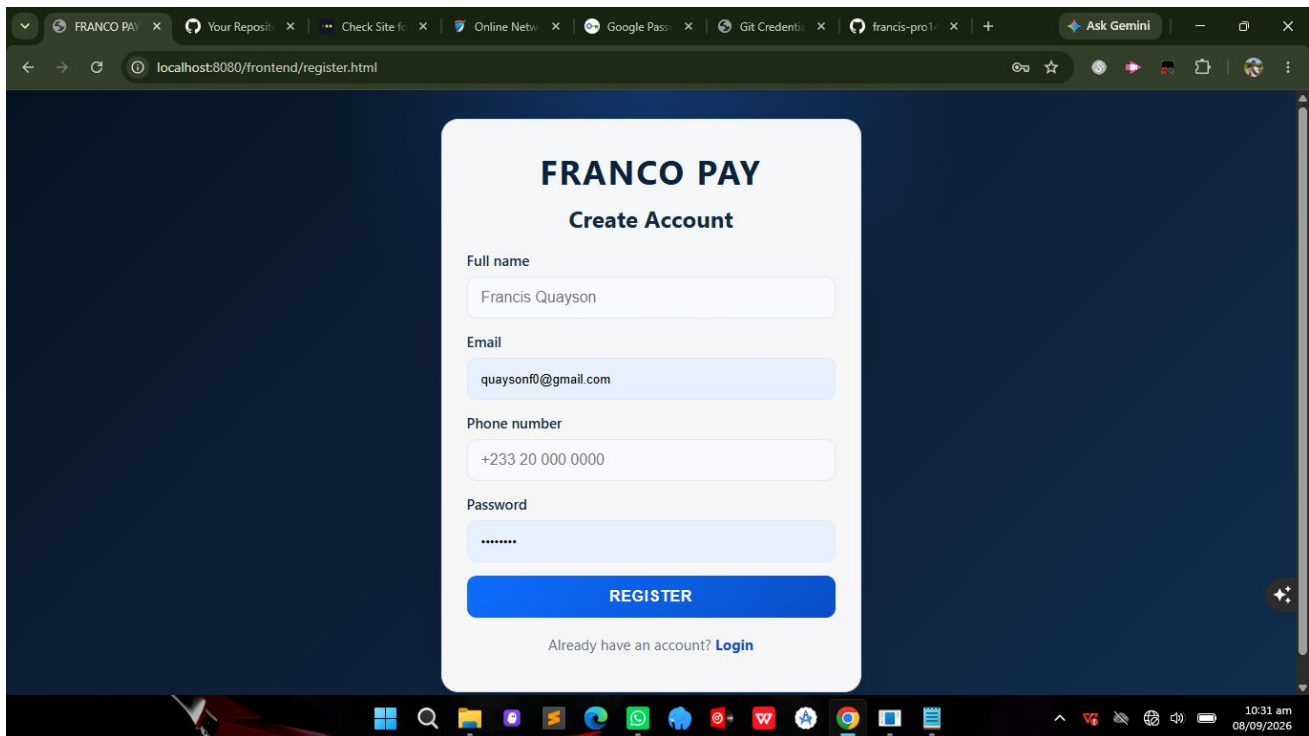
Appendix A — Screenshot Evidence

The actual screenshots from the developed system under the following captions:

A.1 Landing Page



A.2 Registration Page



FRANCO PAY
Create Account

Full name

Email

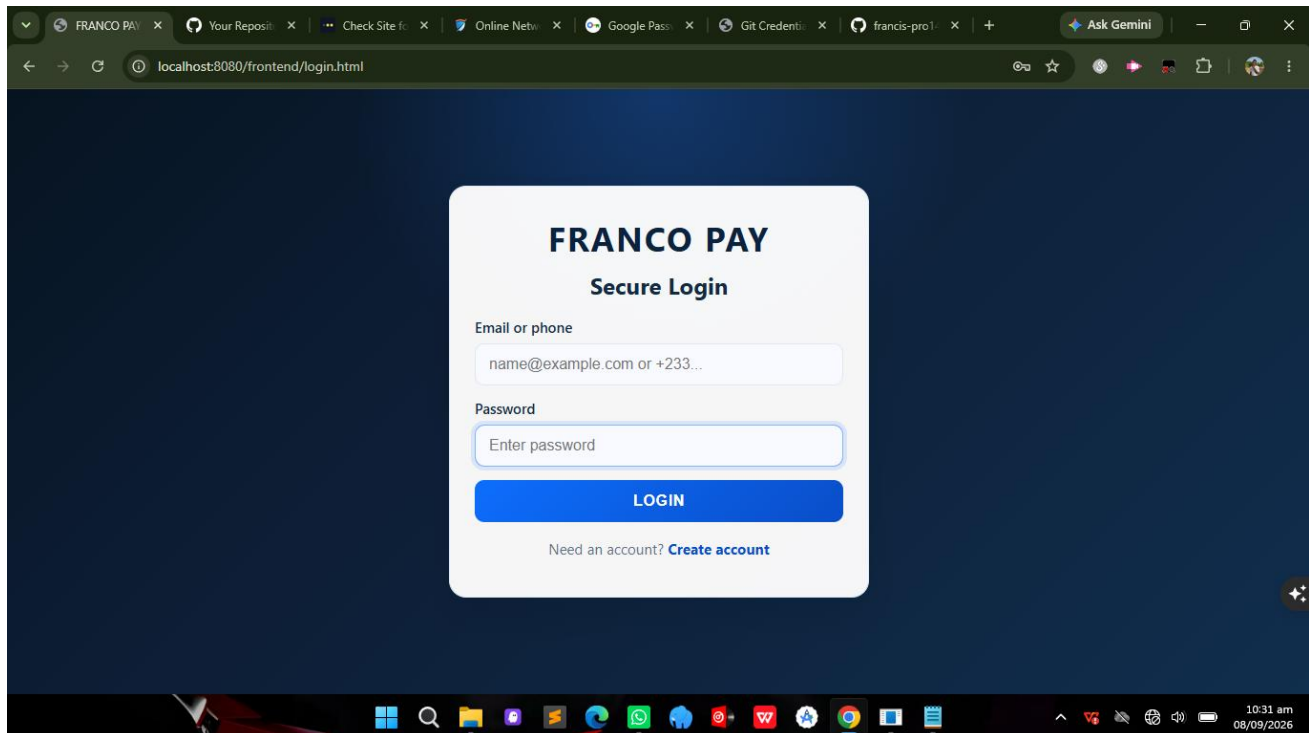
Phone number

Password

REGISTER

Already have an account? [Login](#)

A.3 Login Page



FRANCO PAY
Secure Login

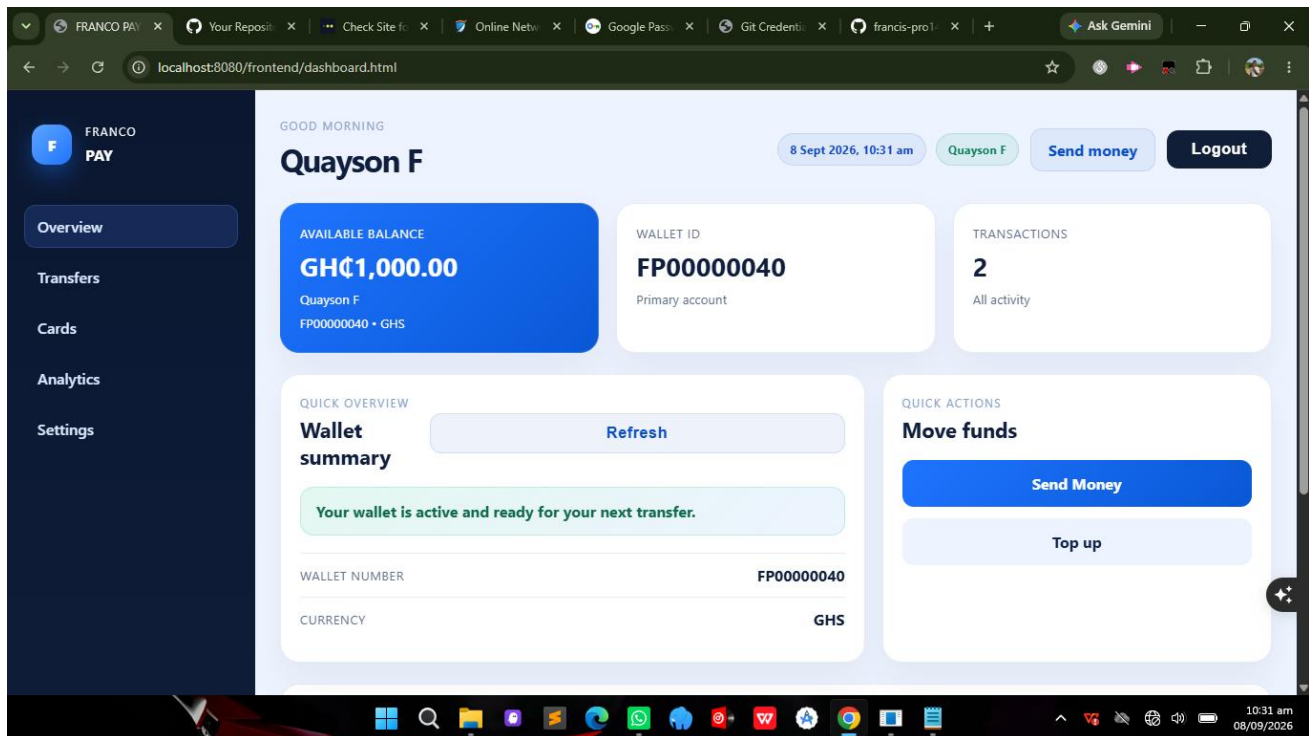
Email or phone

Password

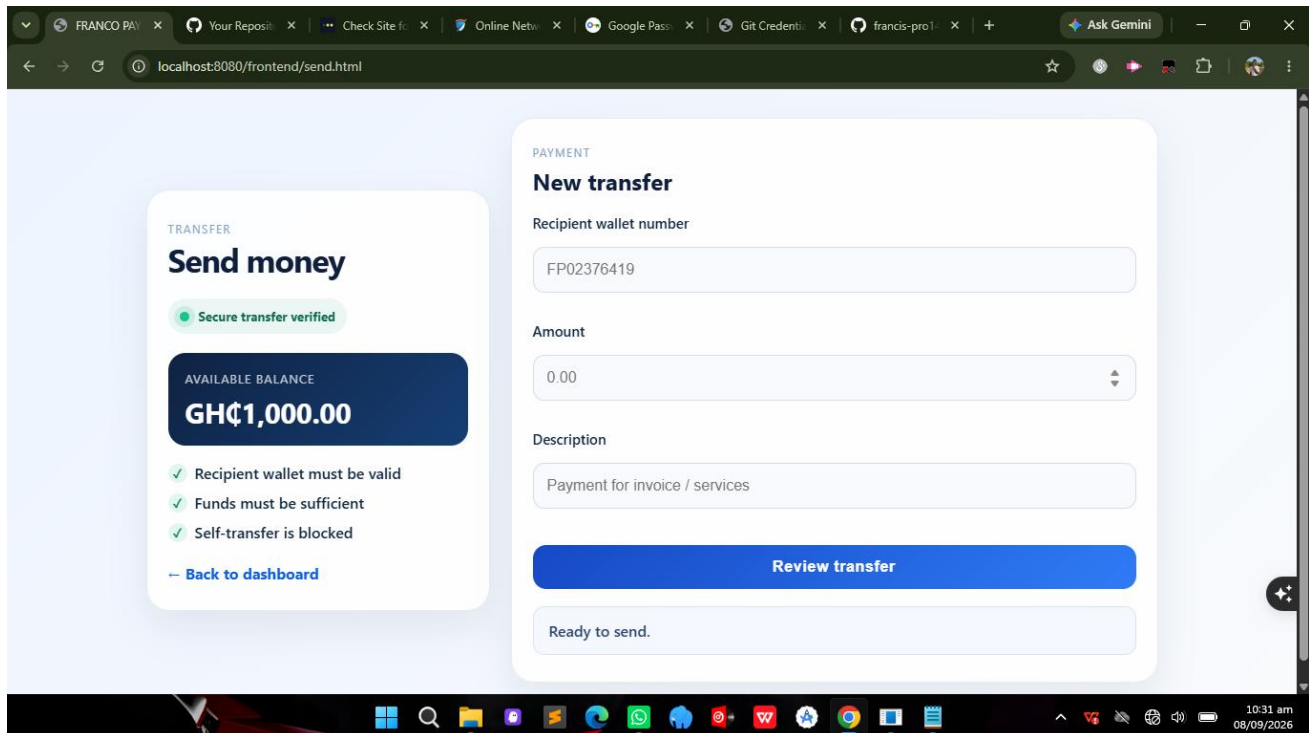
LOGIN

Need an account? [Create account](#)

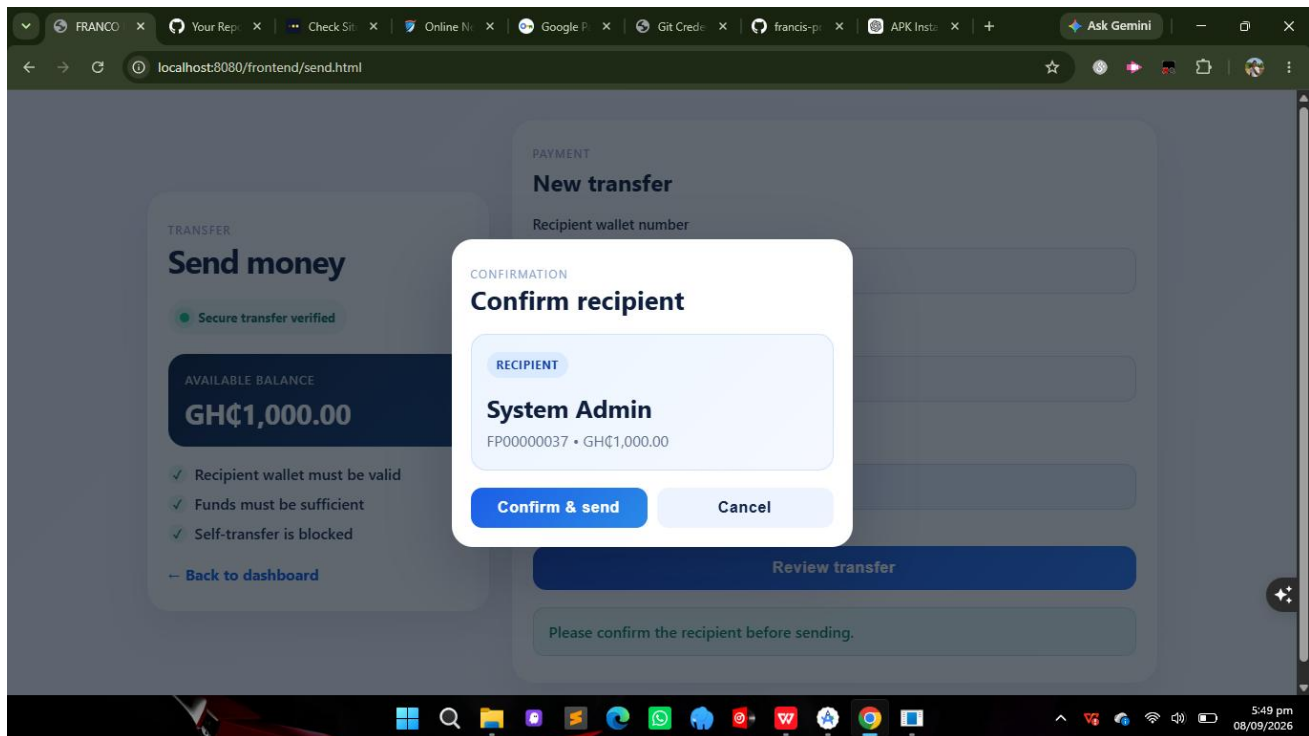
A.4 User Dashboard



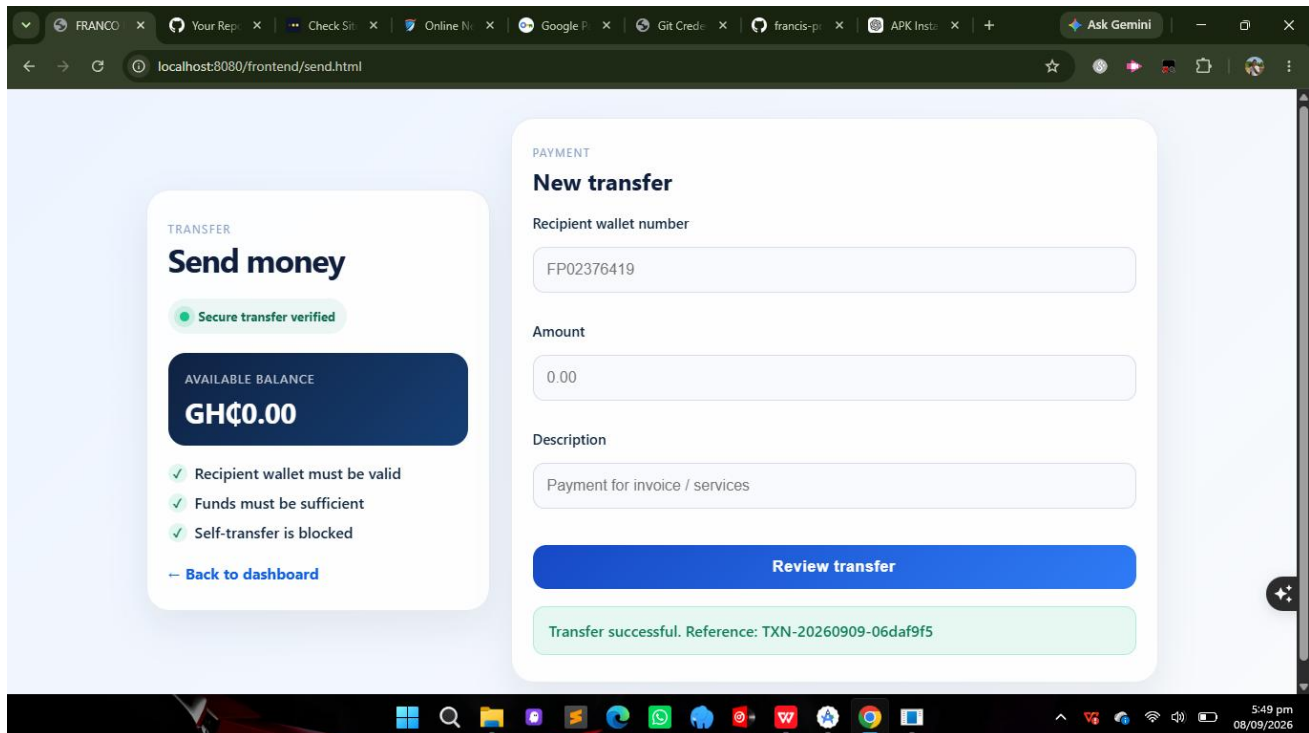
A.5 Send Money Page



A.6 Recipient Confirmation Modal



A.7 Successful Transaction



A.8 Transaction History / Details

